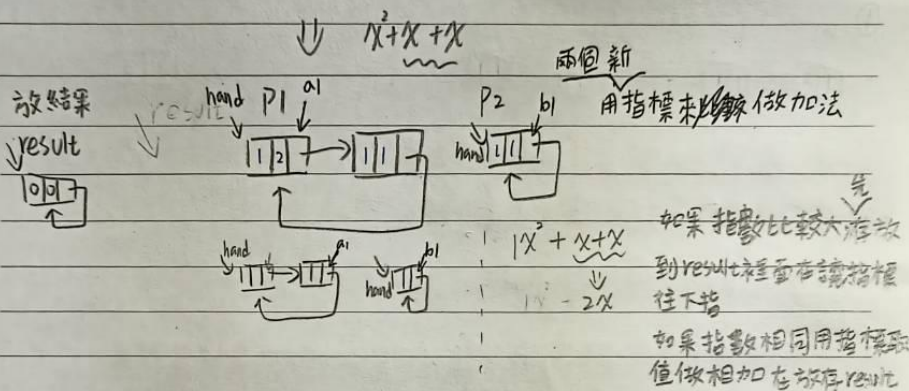
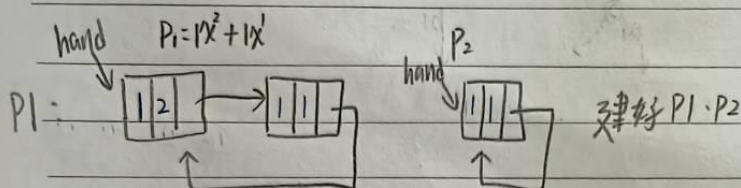
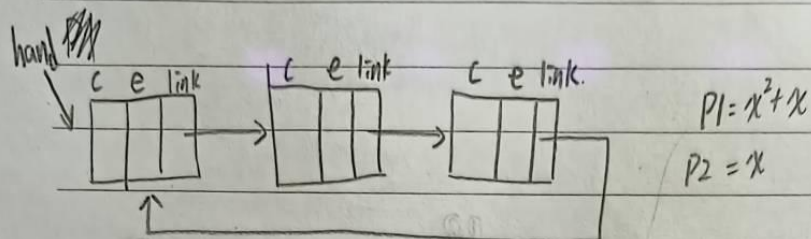


解題思路:

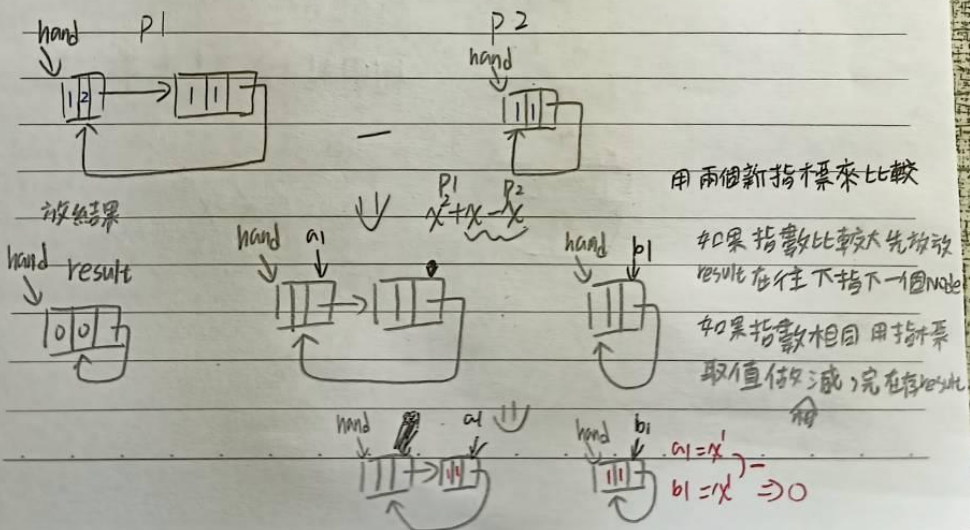
No:

Date: /

加法



減法



No:

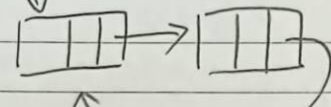
Date:

P1

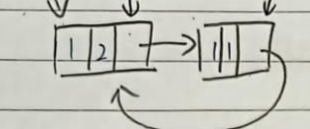
乘法

P2

hand



hand



①

a1

②

b1

112 x 111

111 x 111

用氣泡排序法一個一個下去乘
也是用兩個新指標下去看。

① 乘完的結果放入 result

②

如果有乘出的結果，有指數相同在做相加在放入結果

Eval

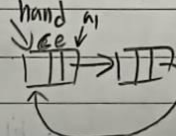
x=1

P1 = x^2 + x

利用指標 a1 來取值

公式

num = C * x^e



P1 = x^2 + x

P2 = x

一般

(x^2 + x) * x

x^3 + x^2

x^3 + x^2

x^3 + x^2

x^3 + x^2

x^3 + x^2

x^3 + x^2

x^3 + x^2

x^3 + x^2

Algorithm Design & Programming

```
Polynomial::~~Polynomial() {
    Node* curr = hand->link;
    while (curr != hand) { // 一個一個解構
        Node* temp = curr;
        curr = curr->link;
        delete temp;
    }
    hand->link = hand;
    delete hand; // 最後解頭
}

Polynomial::Polynomial(const Polynomial &a) {
    hand = new Node{.coef: 0, .exp: -1, .link: NULL};
    hand->link = hand;
    Node *cu = a.hand->link;
    while (cu != a.hand) {
        Added(cu->coef, cu->exp);
        cu = cu->link;
    }
}

void Polynomial::Added(float coef, int exp) {
    Node* prev = hand; // 下一個
    Node* curr = hand->link; // 目前
    while (curr != hand && curr->exp > exp) { // 看要新增哪裡位子
        prev = curr;
        curr = curr->link;
    }
    Node* newTerm = new Node{coef, exp, .link: curr}; // 建立新的節點(1個)
    prev->link = newTerm; // 插入
}
```

```
class Node{
    float coef;
    int exp;
    Node *link; // 指向下一個位子
};

class Polynomial {
private:
    Node *hand;
public:
    Polynomial(); // 建構值
    ~Polynomial(); // 解構值
    Polynomial(const Polynomial& a); // 複製建構值
    void Added(float coef, int exp); // 新增函數
    Polynomial &operator=(const Polynomial &a); // 等號
    Polynomial operator+(const Polynomial &b); // 加法
    Polynomial operator-(const Polynomial &b); // 減法
    Polynomial operator*(const Polynomial &b); // 乘法
    float Eval(float x) const; // 給值代入
    friend ostream& operator<<(ostream& os, const Polynomial& b);
    friend istream& operator>>(istream& is, Polynomial& b);
};

Polynomial::Polynomial() {
    hand = new Node{.coef: 0, .exp: 0, .link: NULL};
    hand->link = hand;
}
```

```

Polynomial &Polynomial::operator=(const Polynomial &a) {
    if (this != &a) { // 要先把等號左邊的值全部消除
        Node* d = hand->link;
        while (d != hand) {
            Node *t = d;
            d = d->link;
            delete t;
        }
        hand->link = hand;

        Node* cu = a.hand->link;
        while (cu != a.hand) { // 放入值
            Added(cu->coef, cu->exp);
            cu = cu->link;
        }
    }
    return *this;
}

Polynomial Polynomial::operator+(const Polynomial &b) {
    Polynomial result;
    Node *a1=hand->link; // 自己
    Node *b1=b.hand->link; // 被加
    while(a1!=hand || b1!=b.hand) {
        if(a1->exp > b1->exp) { // a1比較大放入result
            result.Added(a1->coef, a1->exp);
            a1=a1->link; // 在指向下一個
        }
        else if(a1->exp < b1->exp) { // b1比較大放入result
            result.Added(b1->coef, b1->exp);
            b1=b1->link; // 在指向下一個
        }
    }
}

```

```

91     ✓ else { //指數相同相加完放入result，在兩個指標往下走
92         result.Added(coef: a1->coef+b1->coef, a1->exp);
93         a1=a1->link;
94         b1=b1->link;
95     }
96 }
97 return result;
98 }
99 → ✓ Polynomial Polynomial::operator-(const Polynomial &b) {
100     Polynomial result;
101     Node *a1=hand->link; //自己
102     Node *b1=b.hand->link; //被減
103     ✓ while(a1!=hand || b1!=b.hand) {
104     ✓     if(a1->exp>b1->exp) { //a1比較大放入result
105         result.Added(a1->coef, a1->exp);
106         a1=a1->link; //在指向下一個
107     }
108     ✓ else if(a1->exp<b1->exp) { //b1比較大放入result
109         result.Added(-b1->coef, b1->exp);
110         b1=b1->link; //在指向下一個
111     }
112     ✓ else { //指數相同相減完放入result，在兩個指標往下走
113         result.Added(coef: a1->coef - b1->coef, a1->exp);
114         a1=a1->link;
115         b1=b1->link;
116     }
117 }
118 return result;

```

```

120 → ∨ Polynomial Polynomial::operator*(const Polynomial &b) {
121     Polynomial result;
122     Node *a1=hand->link;//自己
123     ∨ while (a1!=hand) {利用氣泡排序法
124         Node *b1=b.hand->link;//被乘
125         ∨ while(b1!=b.hand) {
126             Node *r=result.hand->link;//檢查有沒有重複的指標
127             float newcoef=a1->coef*b1->coef;
128             int newexp=a1->exp+b1->exp;
129             bool t=false;
130             ∨ while(r!=result.hand) {檢查有沒有重複
131                 ∨ if(r->exp==newexp) {
132                     r->coef+=newcoef;
133                     t=true;
134                     break;
135                 }
136                 r=r->link;
137             }
138             if(!t) result.Added(newcoef,newexp);//沒有重複放入result中
139             b1=b1->link;
140         }
141         a1=a1->link;
142     }
143     return result;
144 }
145 → ∨ float Polynomial::Eval(float x) const {
146     float result=0.0;
147     Node* r=hand->link;
148     ∨ while (r!=hand) {
149         result+=r->coef*pow(x,r->exp);
150         r=r->link;

```



```

151     }
152     return result;
153 }
154 ostream& operator<<(ostream& output, const Polynomial& b) {
155     Node* curr = b.hand->link;
156     bool first = true;
157     while (curr != b.hand) {
158         if (!first && curr->coef > 0) output << "+"; //後一個不會有+
159         output << curr->coef << "x^" << curr->exp;
160         curr = curr->link;
161         first = false;
162     }
163     return output;
164 }
165
166 istream& operator>>(istream& input, Polynomial& b) {
167     int num;
168     cout << "請輸入有幾項指數: ";
169     input >> num;
170     for (int i = 0; i < num; i++) { //一個一個存
171         cout << "請輸入系數與指數(ex:2 3->2x^3): ";
172         int exp;
173         float coef;
174         input >> coef >> exp;
175         b.Added(coef, exp); //新增
176     }
177     return input;
178 }
179

```

效能分析

Time complexity

Added: $O(n)$ ，其中 n 是多項式中項數

多項式加法和減法: $O(n + m)$ ，其中 n 和 m 分別是兩個多項式的項數。

多項式乘法: $O(n * m * k)$ ，其中 k 是結果多項式中的平均項數，最差情況是兩多項式項數的乘積

Eval: $O(n)$ ，其中 n 是多項式的項數

輸入輸出: $O(n)$

Space Complexity

多項式存儲: $O(n)$ ，其中 n 是多項式的項數

操作結果: $O(n + m)$ （加法/減法）或 $O(n * m)$ （乘法）。

總時間和空間

時間複雜度： 主要操作（加法、減法、乘法）分別為 $O(n+m)$ 和 $O(n * m * k)$ 。

空間複雜度： 與結果多項式的項數直接相關，最差情況下 $O(n * m)$

測試與驗證：

$$P1 = x^2 + x \quad P2 = 4x^2$$

代入 $x=1$

$$\text{加: } 5x^2 + x \quad \text{減: } x - 3x^2$$

$$P1=2, P2=4$$

$$\text{乘: } 4x^4 + 4x^3$$

```
請輸入第一項多項式
請輸入有幾項指數: 2
請輸入系數與指數(ex: 2 3->2x^3): 1 2
請輸入系數與指數(ex: 2 3->2x^3): 1 1
請輸入第二項多項式
請輸入有幾項指數: 1
請輸入系數與指數(ex: 2 3->2x^3): 4 2
p1: 1x^2+1x^1
p2: 4x^2
p1+p2=5x^2+1x^1
p1-p2=-3x^2+1x^1
p1*p2=4x^4+4x^3
請輸入要代入多項式的X=1
p1代入x=1共2
p2代入x=1共4
```


申論及開發報告：

使用鏈表（Linked List）存儲多項式的每一項。每個節點包含係數、指數及指向下一節點的指標。此結構適合處理稀疏多項式，因為僅需存儲非零項，時間與空間效率，加法與減法：線性合併，時間複雜度 $O(n+m)$ 。乘法：需額外檢查重複項，最差情況 $O(n \times m \times k)$ 。空間需求：隨多項式項數線性增長，使用循環鏈表，便於尾部插入及邊界處理；排序效率現狀，新增節點時需檢查順序。改進：若輸入已排序，可省略順序檢查。